Status — one line per numbered result

The short answer. `PaperInventory.md` is the long one and stays as the audit trail; this file is the thing to read first.

Measured on `main` after r0049, all six streams merged: build **1372 jobs, 0 errors, 0 warnings**, `sorry` **0**, `axiom` **0**, `sorryAx` **0**. Package: 124 modules, 46,097 lines, 2,119 theorems.

A standing check now watches this file's class of defect. `scripts/a8-claim-check.sh` asks three questions of the elaborated environment — dangling citation, proof-claim against the environment, absence-claim locus — and prints only `NEW`/`RESOLVED` against a committed baseline of **74 accepted obligations**, exiting 1 when `NEW` is non-empty. Cost: **12.3 s wall, 1974 MiB peak RSS**, about 5% of a cold build. It exists because r0046 measured that **seven of eight false proof-claims were true when written**: staleness produces no `sorry` and no build failure, so nothing else signals it. The convention for the scope-qualified sentences it checks is `docs/ScopedClaims.md`.

Labels

Every paper property carries one of five labels. They answer two questions — **is it stated in Lean, and is it proved** — and the pair matters because **`sorry` can only see the second.**

#	Label	Stated?	Proved?	Meaning
1	`S+P`	yes, as the paper states it	yes	done
2	**`S+H`**	yes, as the paper states it	**no — unproven here**	a real Lean statement with a hole. `H` is for *hypothesis*: the result is available only conditionally, as a theorem taking the unproven claim as an argument
3	`S≠`	yes, but **not the paper's statement**	yes	a weakening, an added hypothesis, or a deliberate repair of a printed defect
4	`P`	no — prose only	—	asserted in a docstring, never under the kernel
5	`N`	**no statement at all**	—	nothing in Lean mentions it

A word this file does not use: *open*. In mathematics an open problem is one posed in the literature whose truth is not known to anyone, by proof or by refutation. Nothing here is that. Every unproved statement below is one **this development has not proved yet** — Gunter & Scott proved them all in 1990. So this file says **unproven**, and reserves *open* for its real meaning.

Two consequences worth stating once:

- **`sorry` 0 does not mean "everything is proved."** A `sorry` is a hole in a proof someone *started*. Rows 4 and 5 were never started, and row 2's holes are carried as explicit hypotheses rather than as `sorry`, so none of the three appears in a `sorry` count. That is why this file reports rows 8–11 of the counts separately.
- **`S≠` is not automatically a defect.** Six of the 14 are repairs of the paper's own printed errors, which is correct work; 8 are ours. r0049 re-derived the row population against the built environment rather than inheriting r0044's table, and it moved on both counts: four rows left the census (two closed in r0049, two closed in r0047 and never re-counted), and Theorem 26 moved from *ours* to *repair* once the printed statement was refuted.

Counts

#		Count
1	Numbered results in Gunter & Scott	**30, verified** — 16 Theorems, 13 Lemmas, 1 Proposition, 0 Corollaries; numbers 1–30 contiguous, no gap, no repeat
2	**— fully proved**	**25**
3	— partly proved, some part unproven	2 (results 7, 29)
4	**— corrected and proved**: the stated form was false, the corrected form is proved	2 (results 26, 28)
4a	**— corrected, corrected form still unproven**	1 (result 30)
5	**— all 30 are now stated in the tree**	0 missing
6	Paper properties (numbered conjuncts + prose claims)	239
7	— `S+P` stated and proved	169

#		Count
8	— **`S+H` stated, unproven**	**12**
9	— `S≠` stated, but not as the paper states it	14 (**8 ours**, **6 repairs**)
10	— `P` + `N` no Lean statement at all	36
11	`Prop`-valued claims nothing proves	**7** (8 strict)

Rows 8 and 11 are the two kinds of unfinished work, and they are different: row 8 is a real Lean statement this development has not proved; row 11 is a `def` naming a claim **nobody attempted**. **`sorry` 0 sees neither** — which is why both are counted here and neither shows up in a build.

The 30 numbered results

`P` proved · `p` partly proved · `C` corrected and proved · `c` corrected, still unproven · – not in the tree

`C` and `c` are not "refuted." A result is *refuted* only when a statement is false and nothing correct stands in its place. That is true of no result here. In every case below, some statement was false — sometimes the paper's, more often **our transcription of it** — and a corrected statement exists; `C` says the corrected one is proved, `c` says it is not yet. Reporting these as refutations would credit our own transcription errors as findings about the paper.

#	Result		Status
1	Theorem 1	`P`	least fixed point, and below every fixed point
2	**Theorem 2 (Schroder-Bernstein)**	`P`	*"Let S and T be sets. If `f : S → T` and `g : T → S` are injections, then there is a bijection `h : S → T`"* — printed folio 6. **Genuinely absent until r0048**; now `R48.Agent1.theorem2`. The inventory had mapped it to Mathlib's `Function.Embedding.schroeder_bernstein`, but **that is Zermelo's proof from Knaster–Tarski**, where Gunter & Scott derive it from **Theorem 1** — and `FixedPoint.lean` records that neither implies the other. The new proof follows the paper step for step, via the operator `Y ↦ (T − f*(S)) ∪ f*(g*(Y))`
3	Theorem 3	`P`	`theorem3`, `theorem3_existsUnique`
4	Lemma 4	`P`	`NormalSubposet.lean`
5	Lemma 5	`P`	`FinitaryProjection.lean`
6	Theorem 6	`P`	`theorem6`
7	Theorem 7	`p`	sentence 1 proved; sentences 2–3 proved only at the degenerate strength — **every domain has an `EffectivePresentation`** because `Classical.dec` fills its fields. The recursive forms are unproven: `Theorem7ArrowRecursive`, `Theorem7StrictRecursive`. r0049 restated `StepFunctionsDecidable` over `IsStepEnumeration`, quantified existentially, so the claim no longer names a guard its own hypotheses do not determine — `R49.Agent3.stepFunctionsDecidable_of_compactGuard` proves old → new, a **weakening**. r0049 also supplied the recursion theory the residue needed: the `Finset (ℕ × ℕ)` coding is `Primrec`, the normal-subposet search is total, and §3.2's two conditions **decide boundedness and compute the join's index** (`R49.Agent4.computablePred_bddAbove`, `computable_joinIdx`). One item remains: `RecursiveNormal` for `K(D → E)`
8	Lemma 8	`P`	`Product.lean`
9	Lemma 9	`P`	`lem9_*` — **two printed misprints repaired** (items 3 and 5)
10	Lemma 10	`P`	7 conjuncts, `lem10_{prod,smash,sum,separated,lift,strict}`
11	Theorem 11	`P`	`thm11`, with `thm11_converse`
12	Theorem 12	`P`	`thm12` and the three powerdomains
13	Lemma 13	`P`	`lem13_smyth`, `lem13_hoare`
14	Theorem 14	`P`	`thm14`, both directions
15	**Proposition 15**	`P`	*"A bounded complete domain is bifinite"* — printed folio 31. `prop15` at `Skeleton/Section6.lean:134`, proved since that file was written. **It is a Proposition, not a Theorem or Lemma**, which is the whole reason it read as missing: both instruments that produced the gap searched only for `thm|theorem|lem|lemma`
16	Theorem 16	`P`	`thm16`, `thm16_positive`

#	Result		Status
17	Lemma 17	P	10 conjuncts. r0047 **removed `[BoundedComplete β]`** from `lem17_fun` and `lem17_strictFun`
18	Theorem 18	P	`thm18` — closed r0042, `[propext, Classical.choice, Quot.sound]`
19	Lemma 19	P	`lem19`
20	Lemma 20	P	`lem20`
21	Theorem 21	P	`thm21`
22	Theorem 22	P	`thm22`
23	Lemma 23	P	`lem23`
24	Lemma 24	P	`lem24` (Gunter & Scott's; **not** Gunter 1987's Lemma 24, below)
25	Theorem 25	P	`thm25`, `thm25_isUniversal`
26	Theorem 26	C	**the paper's own statement is false** (`R49.Agent7.not_thm26Printed_of_two_zero_arities`), and **proved as repaired** — `thm26`, whose added `hs : ∀ i, 0 < s i` is therefore a **repair of a printed defect, not a defect of ours**. Theorem 26 as printed is false for any signature with **two or more 0-ary slots**, including the paper's own worked `(2,0,0,0,0,0)`. The proof does not use `fst(ψ(x)) = x`: the combinations F_1 … `F_n` are quantified *before* the algebra, so one instance with $o_i = o_j$ forces $F_i = F_j$ and another with $o_i \neq o_j$ forces $F_i \neq F_j$. It grants every printed hypothesis and asks the *weakest* conclusion — an injective homomorphism, with `isSubalgebraOf_range` proving the image is a subalgebra — so stronger readings of "isomorphic" are refuted a fortiori. The argument previously on record at `Combinator.lean:60–72` **is** invalid, as r0044 said: `isAlgEmbedding_const_of_subsingleton` shows two one-point algebras do land on the same subalgebra. The conclusion was right for a reason nobody had given
27	Theorem 27	P	`thm27`
28	Lemma 28	C	**our** statement was false, not the paper's: we quantified over all `U`, and that closure fails (`not_forall_lemma28`, witness `Flat Empty`), and stays false after adding `[Domain U]` and `[BoundedComplete U]`. The paper's own reading — over its own `Dyadic.U` — is **proved**: `lemma28AtU` at `A4PowerdomainRep.lean:347`, all nine conjuncts, no hypotheses. `UniversalForBCD U` blocks only the generalization we invented
29	Theorem 29	p	sentence 1 proved (`thm29`). Sentence 2: **our** transcription dropped the paper's word "domain", and that form is false (`not_thm29Second`); `Thm29SecondAtDomains`, the paper's actual sentence, is **unproven**. r0049 proved the finite half of what it reduces to — **`A∞` is universal for the finite pointed posets** (`R49.Agent5.exists_normal_embedding_Ainf`, new and unconditional), giving `thm29Normal_finiteBasis`. `Thm29Normal` is unproven **only for infinite `K(E)`**
30	Lemma 30	c	**our** universal closure was false (`not_forall_lemma30`); the paper's reading is `Lemma30AtV` and it is **unproven, now at arity 1** — r0049 proved `FpImagesBifinite V` outright (`R49.Agent6.fpImagesBifinite_V`, no hypotheses, no binders), and conjuncts 1–2 now follow from `Thm29SecondAtDomains` alone. `Thm29Normal` is the single remaining named obstruction, which is unproven statement 2 above

Result 15 was never missing — the instruments were. Both the declaration scan and the docstring grep searched only for `thm|theorem|lem|lemma`, and Gunter & Scott number **Propositions in the same sequence**. `scripts/numbered-status.sh` now matches `prop|proposition|cor|corollary` as well; the widening was measured before it was made, and over results 1–30 it adds exactly one hit and no false positive.

Result 2 was genuinely absent, and the two gaps had independent causes: 15 was a grep blind spot, 2 was a real hole. Widening the heading-word set fixes the 15-class defect but **would still have missed 2**.

Results 4, 5 and 8 carry no numbered declaration either, but are each quoted in a module docstring — `NormalSubposet.lean`, `FinitaryProjection.lean`, `Product.lean` — so **a result stated under a descriptive name is the normal case here**, and a missing number is not evidence of a missing proof.

The figure "30 numbered results" is now verified, by extracting every `(Theorem|Lemma|Proposition|Corollary) N` heading from the full text — ASCII headings are unaffected by the Type 3 font defect. Exactly 30, contiguous, no gap and no repeat. The single Proposition among 29 is the entire cause of this round's measurement defect.

Two unproven statements

The whole of the unfinished mathematics is two statements. Everything else listed as unproved is a consequence of one of them, and will follow once it is proved.

#	Unproven statement	What it needs
1	**`RecursiveNormal` for `K(D → E)`** — reached as `ScottHomCRecursive`	Decide whether two basis elements of `D → E` are bounded, and compute where their join is. Everything around it is done: `StepFunctionsDecidable` is restated and discharged from it (`R49.Agent3.stepFunctionsDecidable_of_scottHomC`), and the order test needs no search at all (`R49.Agent4.computablePred_le_stepValues`)
2	**`Thm29Normal` at infinite `K(E)`**	Realize the type over the **tower's** image rather than the η-image, since the copies must nest under `incl`. Finite bases are **proved** (`R49.Agent5.thm29Normal_finiteBasis`)

The other six follow, and are listed only so nobody re-derives them as separate work:

#	Follows from	Consequence
3	1	`Theorem7ArrowRecursive`
4	1	`Theorem7StrictRecursive`
5	1	`PreservesRecursivePresentation` at the arrow — proved at `fstOp/sndOp`, and r0047 proved the arrow case equivalent to row 3
6	2	`Thm29SecondAtDomains`
7	2	`Lemma30AtV` — and nothing else, now that `FpImagesBifinite V` is proved
8	7	`Lem30Arrow`

Neither of the two is a hard problem in the sense of being unsolved anywhere: Gunter & Scott proved both in 1990, and row 2's finite half is proved here. They are the two places where this development has not yet done the work.

Separately: eight statements that are not the paper's

Not unproven — **proved, but not as the paper states them.** These are the `S≠` rows attributable to us, and they are a different kind of debt: the Lean statement is stronger-hypothesised or otherwise divergent, so proving it does not discharge the paper's sentence. Five of the eight (`Universality.lem24`, `Universality.thm25`) are **one** obstruction, not five — whether the closure image is algebraic, a closure image on an algebraic lattice being continuous rather than algebraic.

Additional theories

Results from other authors, formalized here because the paper's proofs need them. None is part of Gunter & Scott's 30.

Jung, *Cartesian Closed Categories of Domains*

#	Result	Status
1	Corollary 1.36	**proved** — `JungCor136`, not by Jung's route (which goes through Prop. 1.22 and a retraction pair) but by indexing below `cap e`
2	Theorem 1.37	**proved at `[Domain D]`** — `R45.Agent5.thm137`. Full generality unproven here; Jung proved it
3	Theorem 1.37 for chains	**proved at `[Domain D]`** — `thm137Chains`
4	Proposition 1.22	not needed — the route around it is item 1
5	Theorem 2.1, 2.3	quoted; five printed defects in Jung's write-up recorded

Iwamura and Markowsky

#	Result	Status
1	Iwamura's lemma	**proved** — `exists_chain_directed_cover`. **Never in Mathlib**
2	Markowsky's theorem	**proved** — `hasChainSuprema_iff_hasDirectedSuprema`. **Never in Mathlib**
3	`HasWellOrderedInfima`	**no producer exists** — five occurrences, all consumers. A complete reduction chain with an empty left end

Spreen 2005

#	Result	Status
1	Lemma 5.8	**proved** — `PropertyM.hasOmegaOpBoundsAbove_pair`. This, not Iwamura, is what closed Jung's 1.37 here

Gunter 1987, *Universal Profinite Domains*

#	Result	Status
1	Lemma 24 at `M(A)`	**proved** — `R47.Agent1.lemma24_MPair`. **The first proof anywhere**: Gunter's printed Lemma 24 produces *some* `A⁺` and is not about `M(A)`; the `M(A)` form is a remark with no theorem and no proof
2	`HasNormalRealizations` at `Ainf`	**refuted** — `not_hasNormalRealizations_Ainf`. The route to `Thm29Normal` is sound but its hypothesis is unsatisfiable at this tower. **r0049 narrowed what this closes**: the refutation quantifies *universally* over finite normal `A ◃ A∞`, while `Thm29Normal` quantifies *existentially* over embeddings, so it closes **extension** of an embedding already fixed in `A∞` and not **construction** of one. That is what let `lemma24_Step` be run inside the stages
3	Proposition 21, Theorems 22 and 25	used, via the implication `thm29Normal_of_hasNormalRealizations`

Defects in the printed paper

Ten, nine recorded in `StatementRecovery.md`. The tenth is r0049's: **Theorem 26 is false as printed** for any signature with two or more 0-ary slots, kernel-checked by `R49.Agent7.not_thm26Printed_of_two_zero_arities`. That row sat in the suspected-and-refuted table below until r0049 because **the reason on record for it was wrong**, which is a separate defect from the claim being wrong.

Two suspected defects remain **our** transcription errors, not the paper's:

#	Suspected	Actually
1	Theorem 29's second sentence false	our transcription dropped "domain" from "any bifinite domain"
2	the step-function enumeration	our guard tests compactness where the paper tests boundedness — and r0049 sharpened this: on the paper's own index the join always exists, so **the paper tests neither**

An **eleventh candidate is not yet recorded**: r0049's agent3 reports that the same folio-12 sentence writes `⨆{f(y) | y ∈ N ∩ ↓x}` where `f` is bound nowhere and `s` is meant. It is not counted above and not yet in `StatementRecovery.md`.

Prose defects, and what r0049 measured about them

The sweep of documentation claims reached 264 sites. The useful result is a **negative** one: 183 was the wrong denominator. 85 are statements about the proof script directly below them, already checked by the fact that it elaborates; 7 are Gunter & Scott's own sentences; 114 are scope-qualified (`ScopedClaims.md`); **only 43 are decidable against the environment**, and those are now decided in bulk every round rather than by hand.

Of the 18 adjudicated, **9 came out false**. Among the absence claims — sentences asserting the package lacks something — the enrichment is sharp: **7 of the 8 package-scoped rows are false, 87.5% against r0046's 20.8% base rate**. The best specimen is `PropertyM.lean:845`, refuted by a declaration 100 lines below it *in its own file*, missed by r0046's intra-file detector for one reason — its subject is a noun phrase rather than a backticked name. **0 of 55 absence claims named a scope before r0049.**

Two unfixed defects in the instruments themselves, both measured rather than suspected:

#	Instrument	Defect
1	`a8-claim-check.sh`	the baseline keys obligations on `file:line`, so an insertion above a standing obligation reports it as `NEW` + `RESOLVED`. First post-merge run: 1 of each, and they are the **same** claim, moved from `FunctionSpace.lean:527` to `:617` by a docstring edit. Key on normalized claim text instead
2	`a5-r47-conditional.sh`	hardcoded `ROOT` to the `agent5` worktree, so every run after r0047 measured that checkout rather than the caller's. Fixed in r0049 to resolve from the script's own location; **row 8 has still not been re-derived since r0047**

Reproducing

```
scripts/counts.sh                   # modules, lines, theorems, sorry
scripts/numbered-status.sh          # declarations per numbered result
scripts/a6-env-scan.sh <out>        # then a6-summarize.py for row 11
scripts/a5-r47-conditional.sh       # row 8, the S+H conditional surface
scripts/a8-claim-check.sh           # standing prose-staleness check
scripts/compile.sh -r <round>       # build
```

`counts.sh`, `numbered-status.sh` and `a5-r47-conditional.sh` are bash; `axioms.sh`, `a6-env-scan.sh` and `md2pdf.sh` are **zsh** and fail under bash — `${0:A:h}` and unquoted-array word-splitting are zsh-only.